

Progetto reti logiche - Filtro Esponenziale

Alessandro Boglioli

2025

1 Introduzione

Si progetti un circuito sequenziale per la realizzazione di un filtro esponenziale del secondo ordine. Il sistema riceve una sequenza continua di campioni X_t rappresentati in virgola fissa su 32 bit in cui 16 bit rappresentano la parte frazionaria e produce in uscita il segnale filtrato Y_t rappresentato con la stessa notazione e calcolato come:

$$Y_t = \alpha X_t + \alpha(1 - \alpha)Y_{t-1} + (1 - \alpha)^2 Y_{t-2}$$

Per semplicità si limita il coefficiente del filtro α alle potenze negative del 2, cioè:

$$\alpha = \frac{1}{2^k}$$

In questo modo il calcolo dei coefficienti α , $\alpha(1 - \alpha)$ e $(1 - \alpha)^2$ risulta notevolmente semplificato. Il valore k viene fornito in ingresso al sistema e si suppone costante durante l'elaborazione di una sequenza di ingressi. Data la precisione della rappresentazione di ingresso, si limita il valore di k all'intervallo $[0; 7]$. Il comportamento del filtro non è definito se il coefficiente k viene cambiato durante la normale operatività del filtro. Il sistema deve altresì disporre di un segnale di "inizializzazione" che riporta lo stato del filtro a zero.

Una volta realizzato il componente, è richiesto di realizzare un test-bench per la simulazione e la verifica del corretto funzionamento nei diversi casi. In particolare, è necessario verificare il comportamento del filtro rispetto alla risposta al gradino. La simulazione deve tenere conto dei ritardi reali del sistema (simulazione Post Place & Route).

Iniziamo elencando quali sono le ipotesi e assunzioni che ci poniamo per la risoluzione di questo problema:

- Si assume che il valore di α (quindi il valore di k) sia costante durante l'elaborazione di una sequenza
- Il comportamento del sistema non è definito se il valore di α viene modificato durante il funzionamento
- La sequenza di campioni in ingresso può variare in modo asincrono anche durante lo sviluppo di una sequenza di segnali filtrati in uscita
- Gli ingressi, essendo rappresentati senza il bit del segno, sono tutti positivi.

Di seguito vengono riportate alcune limitazioni che andranno ad impattare sullo sviluppo dei risultati:

- La precisione è limitata dalla rappresentazione in virgola fissa a 32 bit, questo genererà delle imprecisioni nei risultati rispetto al mondo reale, in quanto dovremmo accontentarci di una precisione più bassa. Questa limitazione crea discrepanze anche tra soluzioni differenti per lo sviluppo dello stesso filtro esponenziale
- Il sistema dovrà essere sviluppato cercando di minimizzare al massimo i tempi di ritardo e l'area dei vari moduli che lo compongono

2 Specifica

Iniziamo con lo sviluppo della formula con cui si calcola l'uscita Y_t , in modo da individuare un primo approccio da seguire per lo sviluppo del circuito logico.

$$\begin{aligned}Y_t &= \alpha X_t + \alpha(1 - \alpha)Y_{t-1} + (1 - \alpha)^2 Y_{t-2} \\Y_t &= \alpha X_t + (1 - \alpha)[\alpha Y_{t-1} + (1 - \alpha)Y_{t-2}] \\Y_t &= \alpha X_t + (1 - \alpha)[\alpha Y_{t-1} + Y_{t-2} - \alpha Y_{t-2}]\end{aligned}$$

Considerando che la moltiplicazione con α equivale a eseguire uno shift logico, l'ultima equazione sviluppata è un buon punto di partenza per lo sviluppo del circuito, in quanto:

1. Le moltiplicazioni con α possono essere svolte in un solo ciclo di clock tramite un **barrel shifter**.
2. La prima serie di somme/sottrazioni incluse all'interno delle parentesi quadre può essere svolta attraverso un **carry save adder**.
3. La restante porzione dell'equazione può essere calcolata attraverso le operazioni che abbiamo già elencato.

Considerando anche che la sequenza di diversi componenti possa risultare "pesante" per il sistema generale, dividiamo le operazioni grazie all'implementazione di una pipeline tra il calcolo della prima somma e quello della seconda. Quindi, nel nostro caso, il circuito sarà diviso nel calcolo delle seguenti porzioni:

$$\begin{aligned}\text{Sum}_1 &= \alpha Y_{t-1} + Y_{t-2} - \alpha Y_{t-2} \\Y_t &= \alpha X_t + \text{Sum}_1 - \alpha \text{Sum}_1\end{aligned}$$

Come detto in precedenza, questo sistema verrà diviso tramite una pipeline; allo stesso tempo, i valori X_t in ingresso possono variare in modo asincrono. Per questo, dovremo avere a disposizione un modulo per la rivelazione di questa variazione, in modo da poter forzare un reset in modo da calcolare i nuovi valori con gli ingressi corretti.

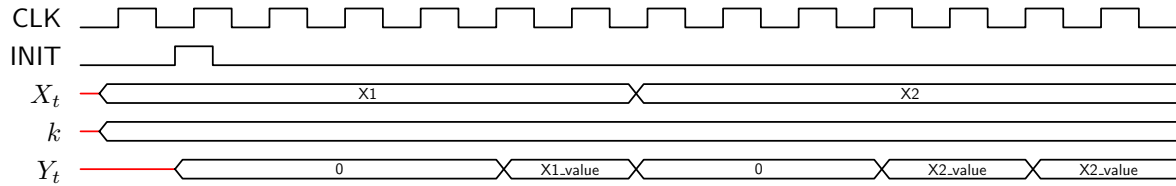
Infine, l'ultimo accorgimento che dobbiamo considerare è la presenza di una sola sottrazione all'interno delle due somme tramite **carry save adder**, la quale può essere semplicemente implementata all'interno dell'apposito modulo.

2.1 Interfaccia del sistema

Il nostro sistema avrà 4 segnali di ingresso e 1 solo segnale di uscita:

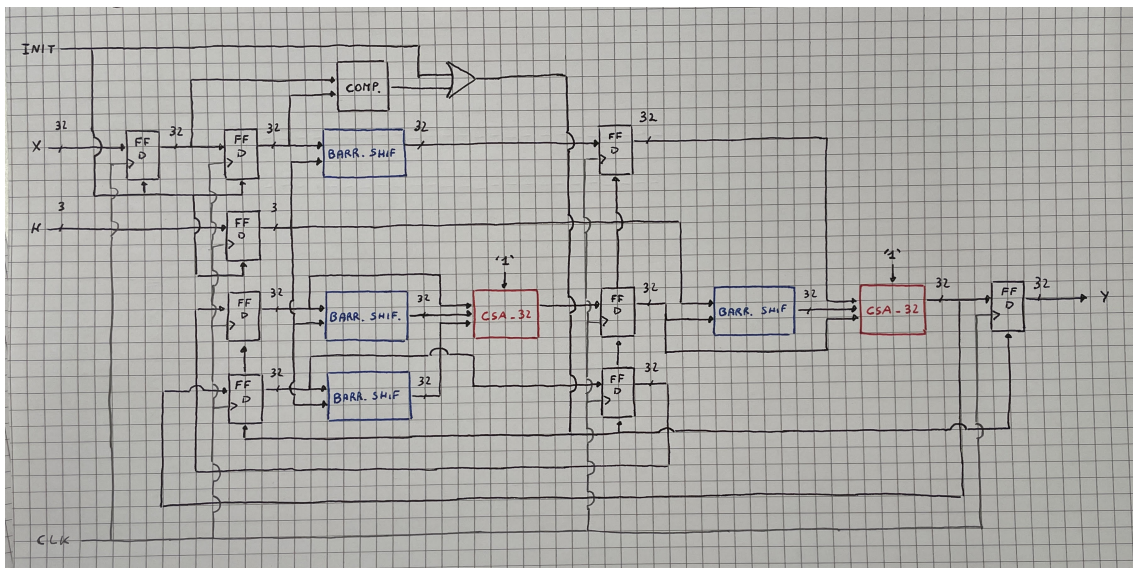
- Segnale X_t :
Segnale di ingresso rappresentato in formato binario su 32 bit, con 16 bit per la parte intera e 16 bit per la parte frazionaria, secondo la codifica in virgola fissa specificata.
- Segnale k :
Segnale di ingresso rappresentato in binario senza segno su 3 bit, sufficiente per rappresentare valori interi compresi tra 0 e 7, corrispondenti all'esponente utilizzato per il calcolo del coefficiente $\alpha = \frac{1}{2^k}$.
- Segnale clk :
Segnale di ingresso su 1 bit che rappresenta il clock del sistema, usato per la sincronizzazione delle operazioni interne.
- Segnale init :
Segnale di ingresso su 1 bit che funge da inizializzatore. Quando attivo, riporta lo stato del filtro a zero.
- Segnale Y_t :
Unico segnale di uscita, anch'esso rappresentato su 32 bit in virgola fissa, con 16 bit per la parte intera. Contiene il valore del segnale in uscita dopo l'elaborazione tramite il filtro esponenziale.

Per meglio rappresentare il legame temporale tra i vari segnali, analizziamo un piccolo diagramma temporale dei segnali:



- Finché INIT non viene posto a 1, il segnale Y_t non potrà mai essere definito, poiché gli stati passati (Y_{t-1} e Y_{t-2}) sono ancora indefiniti.
- Appena INIT viene posto a 1, tutti i registri vengono resettati a 0 in modo asincrono.
- Anche se i valori di X_t e di k erano già all'interno del nostro sistema, per poter produrre una prima uscita bisogna aspettare 4 cicli di clock dopo l'inizializzazione, poiché essi devono essere letti, elaborati e memorizzati prima di poter essere visualizzati.
- I valori calcolati successivamente al primo valore impiegano solamente 2 cicli di clock per essere visualizzati.
- L'ingresso X_t potrebbe variare in modo asincrono durante il calcolo delle uscite, quindi quando esso cambia viene forzato un reset di (quasi) tutti i registri, in modo da ritornare in ad uno stato valido per il calcolo delle uscite.

2.2 Architettura del sistema



Descrizione generale

Il sistema riceve in ingresso tre segnali principali: X , contenente il dato da filtrare, K , un parametro di controllo a 3 bit che regola il coefficiente di attenuazione del filtro, e i segnali di controllo $INIT$ e CLK . L'uscita del filtro è rappresentata da Y .

L'architettura è composta da più sottoblocchi funzionali, riutilizzabili e modulari, tra cui:

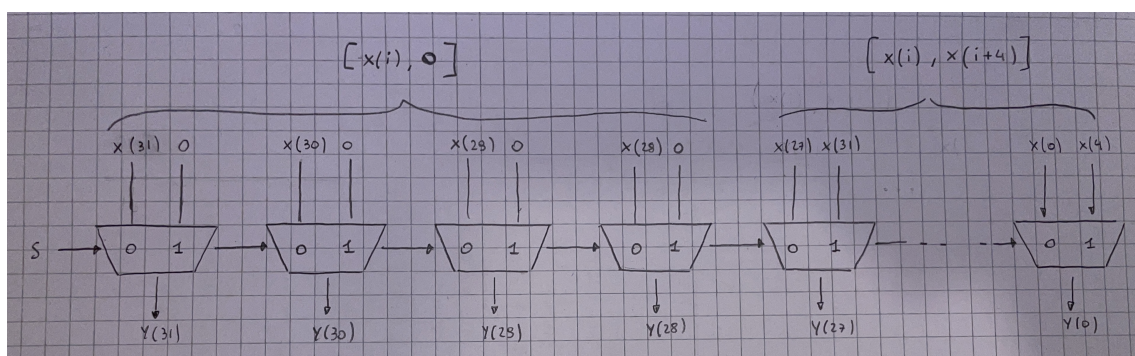
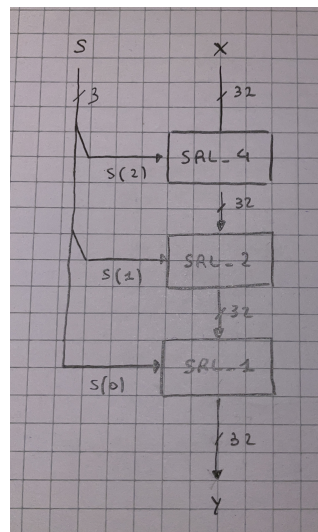
- **Registri a 32 e 3 bit:** impiegati per la sincronizzazione dei segnali X e K e per mantenere i valori intermedi del filtro.
- **Barrel shifter (BARR. SHIF.):** utilizzati per eseguire divisioni rapide per potenze di due, necessarie per il calcolo dei termini shiftati.

- **Carry-save adder (CSA):** adibito alla somma di 3 termini, risparmiando tempo invece di eseguire somme successive.
- **Comparatore e registri della pipeline:** permettono di rilevare cambiamenti in ingresso e sincronizzare l'evoluzione dello stato interno.

Funzionamento del filtro

1. Gli input e gli stati passati vengono registrati all'interno dei registri.
2. Il dato X viene shiftato secondo il valore di K .
3. Le due uscite precedenti vengono anch'esse shiftate e combinate tramite somma.
4. I primi due risultati parziali e l'ultimo stato precedente (Y_{t-1}) vengono registrati all'interno dei registri della pipeline.
5. la somma parziale viene nuovamente shiftata e sommata con la versione shiftata di X .
6. Il valore finale è registrato in Y , rappresentando l'uscita.
7. In caso di variazione dell'ingresso X , il modulo comparatore (COMP.) genera un segnale che disabilita la registrazione dei risultati intermedi nella pipeline.
8. In caso di cambiamento del segnale $INIT$ al valore '1', tutti i registri vengono resettati asincronamente al valore "0".

2.3 Modulo BARREL SHIFTER



Questo modulo è utilizzato per poter svolgere lo shift logico a destra di un ingresso a 32 bit per un massimo di 7 posizioni (valore massimo di k). Per poter effettuare questa operazione in un solo ciclo di clock ci si appoggia a 3 moduli più piccoli, i quali svolgono gli shift logici del numero per valori delle potenze di 2 (2^0 , 2^1 , 2^2). In questo modo possiamo usare i valori dei singoli bit di k per poter controllare l'output in uscita, infatti:

- Se $k = "100"$, si dovrà eseguire uno shift logico di 4 posizioni.
- Se $k = "010"$, si dovrà eseguire uno shift logico di 2 posizioni.
- Se $k = "001"$, si dovrà eseguire uno shift logico di 1 posizione.
- Per tutti gli altri valori di k è possibile eseguire una combinazione dei casi precedenti, mettendoli in serie.

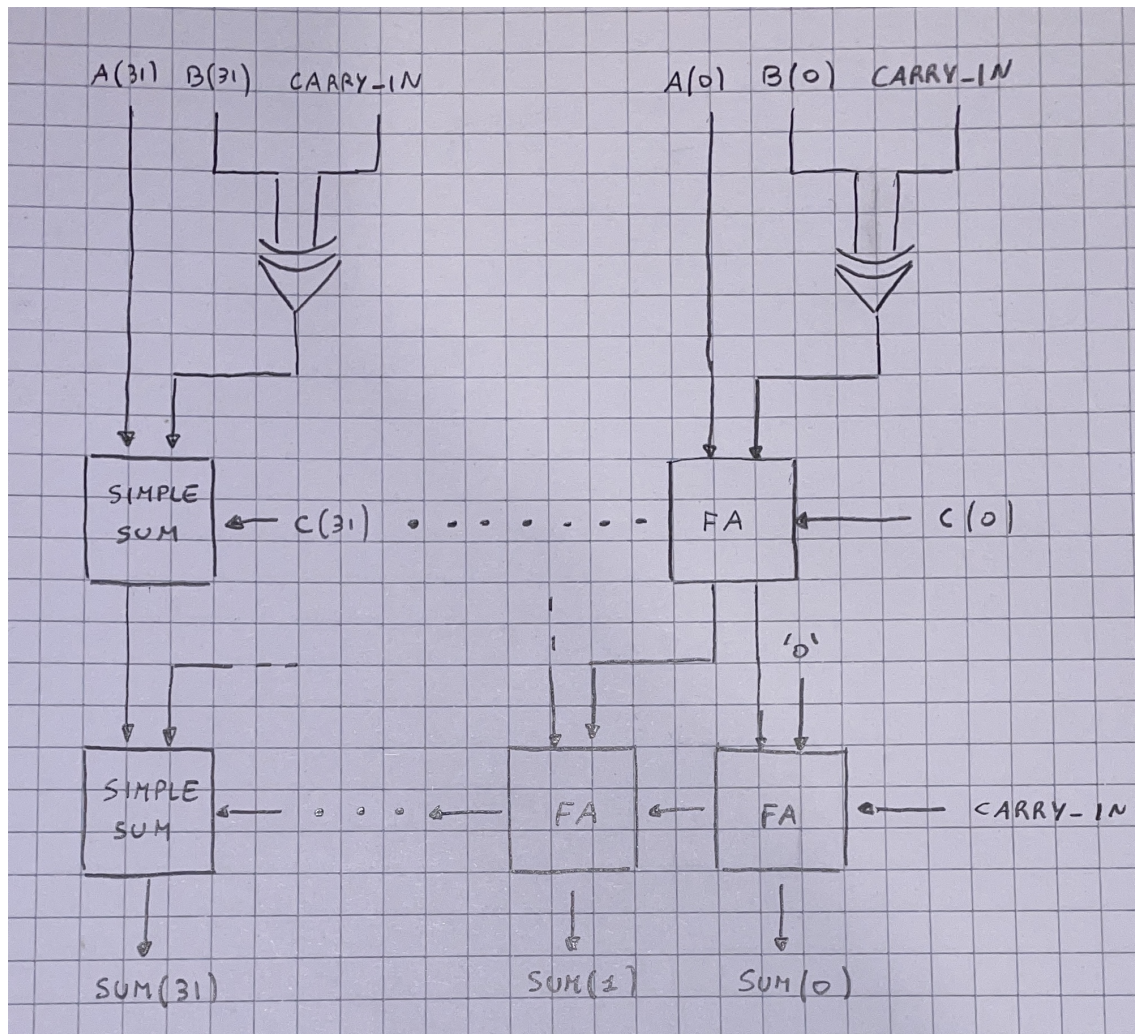
Da questa descrizione si deduce che i 3 moduli ausiliari svolgeranno lo shift logico a destra di 4, 2 e 1 posizione. Questi moduli vengono controllati dai bit dell'ingresso k , rispettivamente:

- Lo shift logico di 4 posizioni viene controllato da $k(2)$.
- Lo shift logico di 2 posizioni viene controllato da $k(1)$.
- Lo shift logico di 1 posizione viene controllato da $k(0)$.

Questi 3 moduli (SRL4, SRL2, SRL1) sono realizzati tutti allo stesso modo, ma differiscono per i collegamenti tra i vari segnali interni. Essi prendono in ingresso un numero di 32 bit e un segnale di controllo ($k(i)$). Per eseguire lo shift, si pongono in parallelo 32 MUX 2 a 1, i quali, pilotati dal segnale di controllo, decidono quale valore far passare al segnale di uscita. Questo è un breve schema di come sono collegati i segnali all'interno dei vari moduli:

- Per SRL4, i primi 4 MUX che decidono la posizione dei bit [28-31] dell'uscita possono scegliere tra i bit dell'ingresso o '0', mentre gli altri MUX possono scegliere tra i bit dell'ingresso non traslati o quelli in 4 posizioni più avanti.
- Per SRL2, i primi 2 MUX che decidono la posizione dei bit [30-31] dell'uscita possono scegliere tra i bit dell'ingresso o '0', mentre gli altri MUX possono scegliere tra i bit dell'ingresso non traslati o quelli in 2 posizioni più avanti.
- Per SRL1, il primo MUX che decide la posizione del bit 31 dell'uscita può scegliere tra il bit originario dell'ingresso o '0', mentre gli altri MUX possono scegliere tra i bit dell'ingresso non traslati o quelli in 1 posizione più avanti.

2.4 Modulo CSA_32



Il modulo CSA_32 implementa un carry-save adder a 32 bit. Esso consente di effettuare l'operazione $A \pm B + C$ in due stadi logici principali: un primo stadio CSA e un secondo stadio di somma ripple-carry. L'operazione effettivamente eseguita dipende dal valore del segnale CARRY_IN:

- Se CARRY_IN = 0, l'unità esegue: $SUM = A + B + C$.
- Se CARRY_IN = 1, l'unità esegue: $SUM = A - B + C$.

Questa differenza è ottenuta invertendo i bit del vettore B tramite un XOR con CARRY_IN, ottenendo così il complemento a uno di B in caso di sottrazione. Il segnale CARRY_IN viene inoltre propagato al secondo stadio RCA come bit di riporto iniziale, completando l'operazione di complemento a due per la sottrazione.

L'architettura si basa su due livelli principali:

1. **Stadio CSA:** utilizza 31 full-adder e un sommatore semplice per calcolare in parallelo le somme parziali tra i bit di A, B_IN ($B \text{ XOR } CARRY_IN$) e C, producendo due vettori: CSA_SUM e CSA_CARRY_OUT.
2. **Modifica segnale:** il segnale RCA_Y, che rappresente uno degli addendi dello stadio RCA, prende come primo bit '0' e per gli altri prende quelli di CSA_CARRY_OUT traslati di una posizione.
3. **Stadio RCA:** i due vettori prodotti dal CSA vengono poi sommati mediante una struttura ripple-carry, sfruttando 31 full-adder e un sommatore semplice alla fine, in quanto non necessitiamo del CARRY_OUT.

2.5 Modulo COMPARATOR

Questo modulo viene utilizzato per rilevare le variazioni asincrone del segnale di ingresso, generando così un segnale di controllo che resetta gli stati del sistema in modo da calcolare i nuovi risultati.

Il modulo prende in ingresso due numeri a 32 bit: A (che rappresenta il nuovo X_t) e B (che rappresenta il suo ultimo valore di X_t registrato). Per identificare una variazione, tutti i bit corrispondenti dei due ingressi vengono sottoposti a un'operazione XOR bit-per-bit. Se due bit corrispondenti sono diversi, il risultato dello XOR sarà '1'. Successivamente, un'operazione OR logico viene applicata a tutti i risultati degli XOR. Il segnale di uscita Y sarà uguale a '1' se è stata rilevata una variazione in qualsiasi bit, indicando che i due numeri in ingresso sono diversi.

Per rendere questa rilevazione sincrona, si deve sincronizzare anche i valori in ingresso in modo che essi non possano cambiare troppo velocemente o troppo vicini al fronte di clock, per questo l'ingresso X passa 2 registri in serie prima di poter essere disponibile al calcolo e i segnali che utilizziamo per il controllo sono proprio le due uscite di questi registri.

3 Verifica

Per la verifica del corretto funzionamento del filtro esponenziale, ho sviluppato due tipi di simulazione:

- **Behavioral simulation** : In questo tipo di simulazione viene verificato il corretto funzionamento del filtro tramite l'analisi degli output e la verifica della loro correttezza. Per questa verifica ci appoggiamo alla generazione dei risultati tramite un piccolo programma C, il quale scrive su un file di testo i risultati attesi che vengono presi da vivado e visualizzati accanto agli output effettivi del sistema durante la simulazione.
- **Post place & route simulation** : In questa simulazione, invece, si andrà a verificare il comportamento del filtro tenendo conto dei ritardi della rete combinatoria e della frequenza del ciclo di clock.

3.1 Test-bench

Per la behavioral simulation, viene istanziato il componente `FILTRO_ESPONENZIALE`, con segnali di ingresso e uscita rappresentati tramite vettori a 32 bit. Il clock è generato con un periodo di 2 ns (valore comodo per la visualizzazione dei risultati, dato che per valori di ingresso alti possono essere impiegate anche diverse interazioni per raggiungere la stabilità), mentre l'ingresso `INIT` viene inizializzato all'inizio della simulazione per impostare lo stato iniziale del filtro.

Un processo dedicato (`LEGGI_E_VERIFICA_Y`) legge dal file di testo i valori di uscita attesi e li confronta con il segnale `Y` prodotto dal filtro. Questo confronto permette di verificare la correttezza del comportamento del filtro nel dominio temporale.

Nella simulazione post place & route, `TB_FILTRO_ESPONENZIALE_PPR`, viene simulata un'esecuzione più vicina alle condizioni reali post-place-and-route, introducendo una fase di attesa iniziale (100 ns) per simulare la fine del GSR (Global Set/Reset). In questo scenario, il segnale di clock viene posto ad un periodo di 8.5 ns, valore ottenuto dall'analisi temporale statica (il critical path ha un tempo di setup pari a 7.125 ns) con un margine di circa il 20%.

3.2 Casi d'uso

I casi d'uso simulati coprono due principali scenari di funzionamento del filtro esponenziale:

- **Verifica funzionale con confronto su file:** nella behavioral simulation si verifica la risposta del sistema dato un ingresso costante e un valore arbitrario per K . I valori di uscita attesi sono precalcolati e memorizzati in un file esterno, che viene letto dinamicamente durante la simulazione per il confronto. Questo permette di testare la convergenza del filtro verso il valore di ingresso. Inoltre si può verificare la correttezza del filtro con valori di ingresso limite, infatti se il valore di X è minore di 2^{-16+k} l'uscita sarà sempre uguale a 0, questo poiché avendo una precisione limitata nel primo calcolo dell'uscita avremo:

$$Y_t = X_t \gg k$$

Dato che i valori Y_{t-1} e Y_{t-2} sono inizializzati a 0

Questo genererebbe $Y_t = 0$ se il valore di di ingresso non è abbastanza grande e dato che i gli stati passati verrebbero aggiornati sempre con il valore 0, non otterremo nessuno sviluppo del risultato.

- **Simulazione PPR temporizzata:** nella simulazione post place & route si riproduce un contesto operativo in cui il filtro viene inizializzato con un valore di ingresso, lasciato evolvere per un certo intervallo di tempo, e poi sottoposto a un azzeramento dell'ingresso o ad una variazione di esso. Questo scenario consente di osservare il comportamento dinamico del filtro a fronte di variazioni rapide dell'input in condizioni di temporizzazione realistiche. Si può notare, infatti, che il segnale di uscita non varia istantaneamente con il ciclo di clock ma ci mette un tempo pari a $t \in [3.302ns; 3.314ns]$ prima di stabilizzarsi completamente.